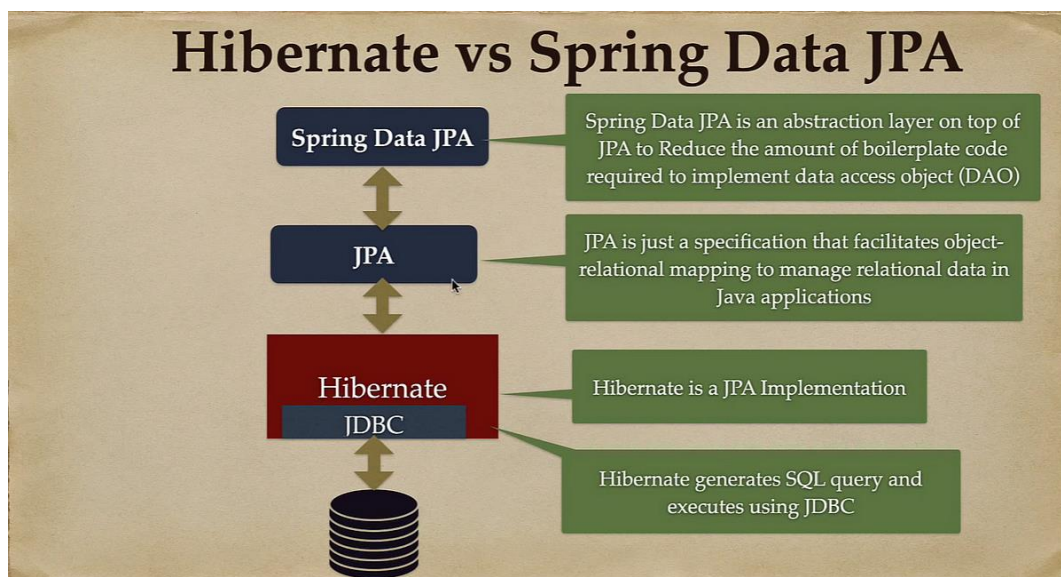
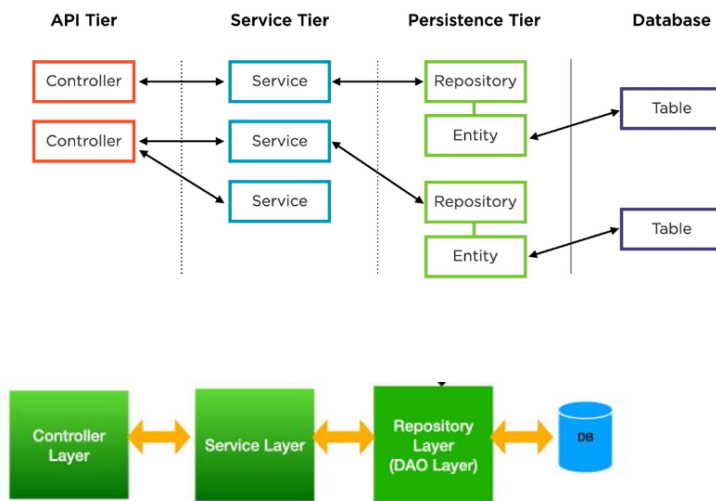


Spring Data JPA

Repository Architectural Overview

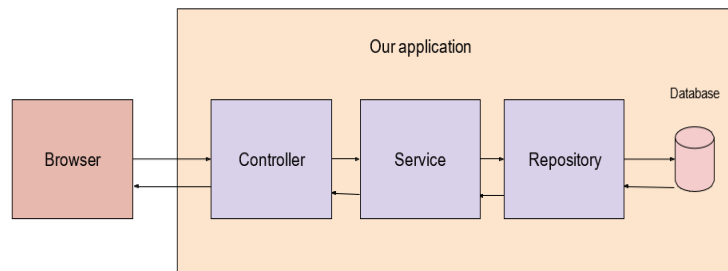


- **Spring Data** is an **umbrella project** that contains multiple sub-projects for data access, supporting multiple types of databases (both relational, NoSQL databases like MongoDB). This reduces the amount of boilerplate code developers need to write.
- **Spring Data JPA** is a *module* within the **Spring Data project** specifically designed to simplify JPA (Java Persistence API) access.
- **JPA** is a specification for data access in Java applications, specifically designed for object-relational mapping (ORM).
- **Hibernate (ORM tool)** is a popular *implementation* of the JPA specification.
- **JPA Repository** simplifies and replaces the need for a custom DAO pattern. still DAO can be used

Example of JPA Providers (Implementations):

- ✓ Hibernate
- ✓ EclipseLink
- ✓ OpenJPA

Spring Data JPA



Spring Data JPA is a **part of the Spring Framework**

it simplifies database operations by providing easy-to-use methods for CRUD, custom queries, and transaction management using JPA (Java Persistence API).

- ✓ **Hibernate** is an **ORM** tool **Object-Relational Mapping**. It is a framework
- ✓ **Java Persistence API (JPA)** is a **specification**
- ✓ **Hibernate** is one of the **implementations** of that specification.
- ✓ **Spring Data JPA** works with **Hibernate** as its default JPA implementation.
- ✓ Purpose: Simplifies database access using JPA with **less boilerplate code** through **repositories** and **derived query methods**.

Advantages

1. **Simplified Data Access:**
 - ✓ **Repository Support:** you can easily create repositories without writing complex **DAO** code. It generates queries automatically based on method names (e.g., `findByName()`).
 - ✓ **No Boilerplate Code:** You don't need to write implementation for standard CRUD operations. (The `JpaRepository` interface provides built-in methods like `save()`, `findAll()`, `findById()`, and `deleteById()`)
2. **Custom Queries:** Allows custom queries using the `@Query` annotation.
3. **Automatic Pagination & Sorting:** Built-in support for pagination and sorting.
4. **Declarative Transactions:** Simplifies transaction management using `@Transactional`.

Note: Additional information

Optimized Performance

- **Caching:** It integrates well with caching solutions like Hibernate's second-level cache to improve performance.
- **Lazy Loading and Eager Loading:** You can control how **related entities are loaded** (lazily or eagerly), optimizing performance based on your requirements.

Steps to Create a Spring Data JPA Application

1. Create a Maven project using **Spring Initializr** by adding the dependencies

1. **Spring Web**
2. **Spring Data JPA**
3. **MySQL Driver**
4. Lombok (Optional)

2. Create Directory Structure:

1. **controller**
2. **service**
3. **repository**
4. **entity**

3. Set Up the Database

1. Create the **database**(e.g. **testdb**)
2. Configure database connection settings in **application.properties**

4. Create an Entity Class

- Define a class annotated with **@Entity** use **@Id** and **@GeneratedValue** to define the primary key.

5. Create a Repository Interface

- Extend the **JpaRepository** interface for the entity to enable **built-in CRUD operations**.

6. Create a Service Layer (*Optional but Recommended*)

- Define a **service interface** and a **ServiceImpl class** which Implement the service interface for business logic

7. Create a Controller

- Create a REST controller to define endpoints for CRUD operations. Use annotations like **@GetMapping**, **@PostMapping**, **@PutMapping**, and **@DeleteMapping**.

8. Run and Test the Application

Start the application and test it with use tools like **Postman**, **cURL**, or a **browser** to test the API endpoints.

// Application Entry Point

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
```

Employee Entity

```
package com.example.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    // @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    // Constructors getters and setters

    public Employee() {
    }

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
```

```

        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

Note: If you're using `@GeneratedValue(strategy = GenerationType.IDENTITY)`, you don't need to pass the id value when creating the employee object in the request. The database will handle it.

//Employee Repository

```

package com.example.repository;

import com.example.demo.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

@Repository //Not needed
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}

```

Employee Controller without Service directly using repo

```

package com.example.controller;

import com.example.demo.entity.Employee;
import com.example.demo.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class EmployeeController {

    @Autowired
    private EmployeeRepository repository;

    // Get all employees
    @GetMapping("/employees")

```

```

public List<Employee> getAllEmployees()
{
    return repository.findAll();
}

// Create a new employee
@PostMapping("/employees")
public Employee createEmployee(@RequestBody Employee employee)
{
    return repository.save(employee);
}

// Get employee by ID
@GetMapping("/employees/{id}")
public Employee getEmployeeById(@PathVariable Long id)
{
    return repository.findById(id).orElseThrow(() -> new RuntimeException("Employee not found"));
}

// Update an existing employee
@PutMapping("/employees/{id}")
public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee updatedEmp)
{
    Employee existingEmp = repository.findById(id).get(); // Assumes the record exists
    existingEmp.setName(updatedEmp.getName());
    return repository.save(existingEmp);
}

// Delete an existing employee
@DeleteMapping("/employees/{id}")
public String deleteEmployee(@PathVariable Long id) {
    repository.deleteById(id);
    return "Employee deleted successfully";
}
}

```

Api endpoints

Operation	HTTP Method	Complete URL	Request Body
Get all employees	GET	/api/employees	None
Create a new employee	POST	/api/employees	{ "name": "Tom" }
Get employee by ID	GET	/api/employees/{id}	None
Update an existing employee	PUT	/api/employees/{id}	{ "name": "Tom Updated" }
Delete an employee	DELETE	/api/employees/{id}	None

If you want to use Service and ServiceImpl

Employee Service Interface

```
package com.example.service;
import com.example.entity.Employee;
import java.util.List;

public interface EmployeeService {

    List<Employee> getAllEmployees();

    Employee getEmployeeById(Long id);
    Employee createEmployee(Employee employee);
    Employee updateEmployee(Long id, Employee updatedEmp);

    void deleteEmployee(Long id);
}
```

// Employee Service Implementation

```
package com.example.service;

import com.example.entity.Employee;
import com.example.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class EmployeeServiceImpl implements EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    @Override
    public List<Employee> getAllEmployees()
    {
        return repository.findAll();
    }

    @Override
    public Employee getEmployeeById(Long id)
    {
        return repository.findById(id).orElseThrow(() -> new RuntimeException("Employee not found"));
        // return repository.findById(id).orElse(null);
    }
}
```

```

@Override
public Employee createEmployee(Employee employee)
{
    return repository.save(employee);
}

public Employee updateEmployee(Long id, Employee updatedEmp)
{
    Optional<Employee> optionalEmp = repository.findById(id);
    if (optionalEmp.isEmpty())
    {
        return null; // Or handle "not found" case differently
    }
    Employee existingEmp = optionalEmp.get();
    existingEmp.setName(updatedEmp.getName());
    return repository.save(existingEmp);
}
}

@Override
public void deleteEmployee(Long id) {
    repository.deleteById(id);
}
}

```

Note: the updateEmployee() can be written like this as well using map() as well

```

@Override
public Employee updateEmployee(Long id, Employee updatedEmp) {
    return repository.findById(id).map(employee -> {
        employee.setName(updatedEmp.getName());
        return repository.save(employee);
    }).orElseThrow(() -> new RuntimeException("Employee not found"));
}

```


// Employee Controller using Service

```
package com.example.controller;

import com.example.entity.Employee;
import com.example.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class EmployeeController {

    // You need to modify the controller injecting the employeeService instead of repository
    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/employees")
    public List<Employee> getAllEmployees() {
        return employeeService.getAllEmployees();
    }

    @PostMapping("/employees")
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeService.createEmployee(employee);
    }

    // Get employee by ID
    @GetMapping("/employees/{id}")
    public Employee getEmployeeById(@PathVariable Long id) {
        return employeeService.getEmployeeById(id);
    }

    @PutMapping("/employees/{id}")
    public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee
updatedEmployee) {
        return employeeService.updateEmployee(id, updatedEmployee);
    }

    @DeleteMapping("/employees/{id}")
    public String deleteEmployee(@PathVariable Long id) {
        employeeService.deleteEmployee(id);
        return "Employee deleted successfully";
    }
}
```

Controller Using ResponseEntity with

```
package com.example.demo.controller;
import com.example.demo.entity.Employee;
import com.example.demo.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class EmployeeController {

    @Autowired
    private EmployeeService employeeService;

    // Get all employees
    @GetMapping("/employees")
    public ResponseEntity<List<Employee>> getAllEmployees() {
        List<Employee> employees = employeeService.getAllEmployees();
        return ResponseEntity.ok(employees); // HTTP 200 OK which is default
    }

    // Get an employee by ID
    @GetMapping("/employees/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable Long id) {
        Employee employee = employeeService.getEmployeeById(id);
        return ResponseEntity.ok(employee);
    }

    // Create a new employee
    @PostMapping("/employees")
    public ResponseEntity<Employee> createEmployee(@RequestBody Employee employee) {
        Employee createdEmployee = employeeService.createEmployee(employee);
        return ResponseEntity.status(201).body(createdEmployee); // HTTP 201 Created
        //return ResponseEntity.status(HttpStatus.CREATED).body(createdEmployee); // HTTP 201 Created
    }

    // Update an existing employee
    @PutMapping("/employees/{id}")
    public ResponseEntity<Employee> updateEmployee(@PathVariable Long id, @RequestBody
    Employee updatedEmployee) {
        Employee employee = employeeService.updateEmployee(id, updatedEmployee);
        return ResponseEntity.ok(employee);
    }

    // Delete an employee
```

```

@DeleteMapping("/employees/{id}")
public ResponseEntity<String> deleteEmployee(@PathVariable Long id) {
    employeeService.deleteEmployee(id);
    return ResponseEntity.ok("Employee deleted successfully.");
    //return ResponseEntity.noContent().build();
}
}

```

Ideal API Design Based on the Code

Operation	HTTP Method	Complete URL	Request Body	Response Code
Get all employees	GET	http://localhost:8080/api/employees	None	200 OK
Create a new employee	POST	http://localhost:8080/api/employees	{ "name": "Tom" }	201 Created
Get employee by ID	GET	http://localhost:8080/api/employees/{id}	None	200 OK (if found), 404 Not Found (if not found)
Update employee by ID	PUT	http://localhost:8080/api/employees/{id}	{ "name": "Tom Updated" }	200 OK (if found), 404 Not Found (if not found)
Delete employee by ID	DELETE	http://localhost:8080/api/employees/{id}	None	204 No Content (if successfully deleted), 404 Not Found (if not found)

Some method of Repository

Method	Description	Example
save (S entity)	Saves or updates an entity.	employeeRepository.save(employee);
findById (ID id)	Retrieves an entity by its ID.	employeeRepository.findById(1L);
findAll ()	Retrieves all entities from the database.	employeeRepository.findAll();
deleteById (ID id)	Deletes an entity by its ID.	employeeRepository.deleteById(1L);
count ()	Returns the total number of entities.	employeeRepository.count();
existsById (ID id)	Checks if an entity with the given ID exists.	employeeRepository.existsById(1L);

Additional information:

Note: There are different ways to write updateEmployee() method

1) Using null checking

```
public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee updatedEmp) {
    Employee existingEmp = repository.findById(id).orElse(null); // Use orElse(null) to avoid exception
    if (existingEmp != null) {
        existingEmp.setName(updatedEmp.getName());
        return repository.save(existingEmp);
    }
    return null; // Return null if not found
}
```

2) Using map()

```
@PutMapping("/{id}")
public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee
updatedEmp) {
    return repository.findById(id).map(employee -> {
        employee.setName(updatedEmp.getName());
        return repository.save(employee);
    }).orElseThrow(() -> new RuntimeException("Employee not found"));
}
```

3) Using optional

```
public Optional<Employee> updateEmployee(@PathVariable Long id, @RequestBody Employee
updatedEmp) {
    Optional<Employee> optionalEmp = repository.findById(id);
    if (optionalEmp.isPresent()) {
        Employee employee = optionalEmp.get();
        employee.setName(updatedEmp.getName());
        return Optional.of(repository.save(employee));
    }
    return Optional.empty();
}
```